

# IAM-03: Group Architecture and Design

**Series:** IAM — IAM Administration | **Notebook:** 3 of 12 | **Created:** January 2026 | **Last Updated:** 08/12/2026

## Designing Scalable Group Structures

Groups are the foundation of access management. A well-designed group architecture simplifies administration, improves security, and scales with your organization.

## Table of Contents

- 1. [Group Fundamentals](#)
- 2. [Group Hierarchy Patterns](#)
- 3. [Naming Conventions](#)
- 4. [SAML Group Mapping](#)
- 5. [Group Lifecycle Management](#)
- 6. [Separation of Duties](#)
- 7. [Cross-Environment Patterns](#)
- 8. [Group Assessment Queries](#)

## Prerequisites

| Requirement           | Details                    |
|-----------------------|----------------------------|
| Dynatrace Environment | SaaS with Gen3 IAM enabled |

| Requirement     | Details                                          |
|-----------------|--------------------------------------------------|
| Permissions     | account-iam-admin or group management rights     |
| Prior Knowledge | <b>IAM-01</b> and <b>IAM-02</b> (SSO configured) |

## 1. Group Fundamentals

Groups in Dynatrace Gen3 IAM are collections of users that share access permissions. Groups exist at the **account level** and can be assigned to one or more environments.

### Group Properties

| Property            | Description                       | Required    |
|---------------------|-----------------------------------|-------------|
| <b>Name</b>         | Unique identifier for the group   | Yes         |
| <b>Description</b>  | Purpose and ownership information | Recommended |
| <b>Owner</b>        | Person responsible for membership | Recommended |
| <b>Federated ID</b> | SAML group mapping (if SSO)       | Optional    |

### What Groups Control

Groups are assigned:

- **Account-level policies** (apply to all environments)
- **Environment-level policies** (scoped to specific environments)
- **Boundaries** (what entities they can see)

### Group vs User Assignment

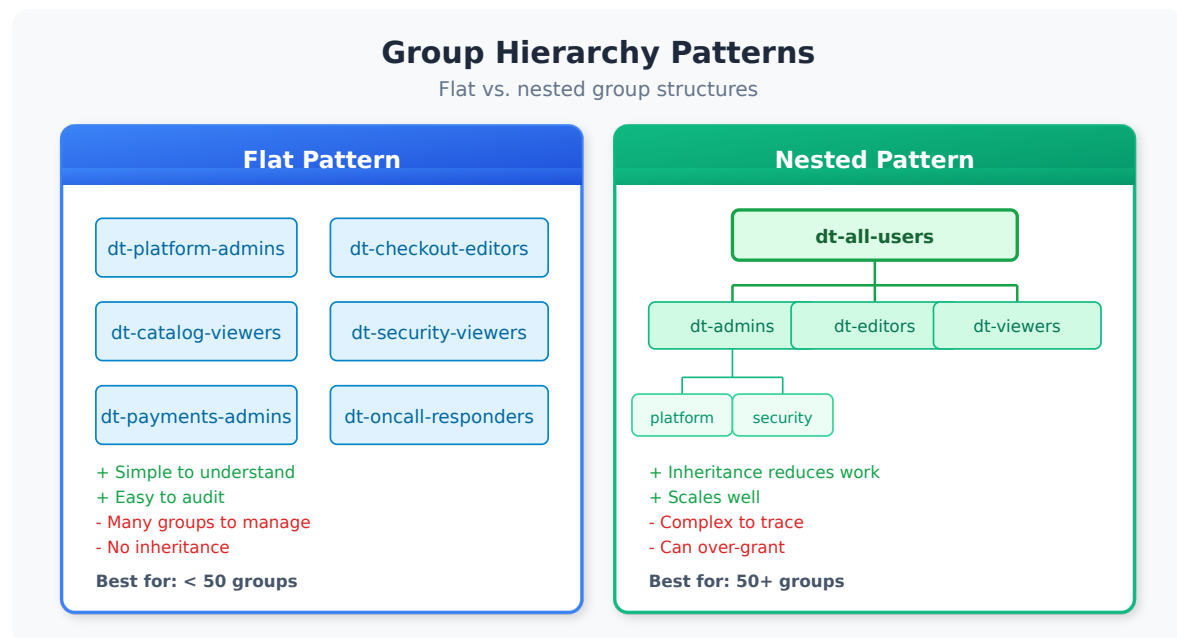
| Approach    | Use When                      | Maintenance                    |
|-------------|-------------------------------|--------------------------------|
| Group-based | Standard case for all users   | Low - manage membership only   |
| Direct user | Break-glass, temporary access | High - track individual grants |

**Best Practice:** Always prefer group-based access. Direct user assignment should be

rare and time-limited.

## 2. Group Hierarchy Patterns

Choose a structure that matches your organization and scales appropriately.



### Pattern 1: Flat Structure

All groups exist at the same level with no hierarchy.

```
Groups:
├─ platform-admins
├─ platform-viewers
├─ team-a-users
├─ team-b-users
└─ auditors
```

**Pros:** Simple, easy to understand

**Cons:** Doesn't scale, no inheritance

### Pattern 2: Role-Based

Groups organized by function/role.

Groups:

|               |                     |
|---------------|---------------------|
| — dt-admins   | (full access)       |
| — dt-editors  | (read + write)      |
| — dt-viewers  | (read only)         |
| — dt-auditors | (compliance access) |

**Pros:** Clear permission levels

**Cons:** Limited data segregation

### Pattern 3: Team-Based

Groups organized by team ownership.

Groups:

|                 |                           |
|-----------------|---------------------------|
| — checkout-team | (checkout service owners) |
| — payments-team | (payments service owners) |
| — platform-team | (infrastructure owners)   |
| — sre-oncall    | (incident response)       |

**Pros:** Maps to ownership model

**Cons:** Permission levels mixed within groups

### Pattern 4: Matrix (Recommended for Enterprise)

Combines role and team dimensions.

Groups:

|                    |                               |
|--------------------|-------------------------------|
| — checkout-admins  | (checkout team, admin access) |
| — checkout-editors | (checkout team, edit access)  |
| — checkout-viewers | (checkout team, view access)  |
| — payments-admins  |                               |
| — payments-editors |                               |
| — payments-viewers |                               |
| — platform-admins  | (cross-cutting admin)         |
| — all-viewers      | (everyone, read-only)         |

**Pros:** Fine-grained control, scales well

**Cons:** More groups to manage

## Choosing a Pattern

| Organization Size | Recommended Pattern  |
|-------------------|----------------------|
| < 50 users        | Flat or Role-Based   |
| 50-500 users      | Team-Based or Matrix |
| > 500 users       | Matrix               |

## 3. Naming Conventions

Consistent naming makes groups discoverable and self-documenting.

### Recommended Format

```
<scope>;-<team/domain>;-<role>;
```

#### Examples:

| Group Name          | Scope     | Team     | Role           |
|---------------------|-----------|----------|----------------|
| dt-checkout-admins  | Dynatrace | Checkout | Admin          |
| dt-payments-viewers | Dynatrace | Payments | Viewer         |
| dt-platform-editors | Dynatrace | Platform | Editor         |
| dt-all-oncall       | Dynatrace | All      | On-call access |

## Naming Rules

| Rule                          | Rationale                        |
|-------------------------------|----------------------------------|
| Use lowercase                 | Consistency, SAML compatibility  |
| Use hyphens (not underscores) | URL-safe, readable               |
| Prefix with dt -              | Identify Dynatrace groups in IdP |
| Keep under 50 characters      | Display limits                   |

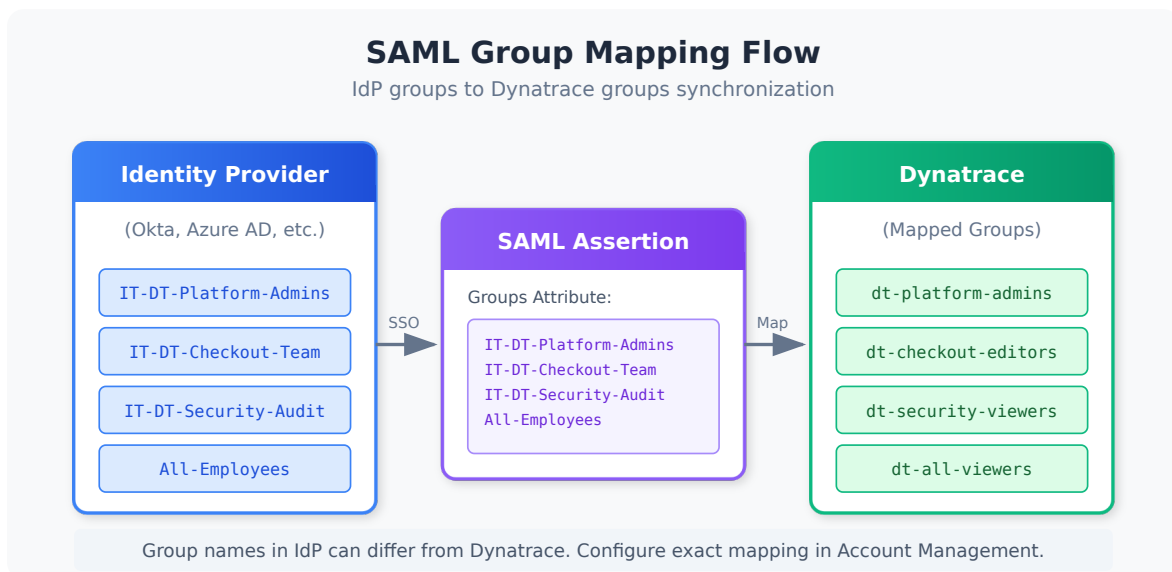
| Rule                | Rationale            |
|---------------------|----------------------|
| Avoid abbreviations | Clarity over brevity |

## Anti-Patterns to Avoid

| Bad Name        | Problem               | Better Name        |
|-----------------|-----------------------|--------------------|
| Team A          | Spaces, generic       | dt-team-a-users    |
| ADMINS          | Uppercase, no context | dt-platform-admins |
| grp_dt_chk_adm  | Cryptic abbreviations | dt-checkout-admins |
| dynatrace-users | Too generic           | dt-all-viewers     |

## 4. SAML Group Mapping

Federating groups from your Identity Provider (IdP) automates user provisioning and ensures consistent access.



## How It Works

1. User authenticates via SAML SSO
2. IdP includes group memberships in SAML assertion
3. Dynatrace maps IdP groups to Dynatrace groups

4. User receives permissions from mapped groups

## Mapping Strategies

| Strategy              | Description                        | Use When                |
|-----------------------|------------------------------------|-------------------------|
| <b>1:1 Mapping</b>    | IdP group = DT group               | Simple, clear ownership |
| <b>Many:1 Mapping</b> | Multiple IdP groups → one DT group | Consolidating access    |
| <b>1:Many Mapping</b> | One IdP group → multiple DT groups | Granting multiple roles |

## IdP Group Attribute

Configure your IdP to send groups in the SAML assertion:

### Azure AD (Entra ID):

```
Claim name: groups
Source attribute: user.assignedgroups
```

### Okta:

```
Attribute: groups
Value: Matches regex: dt-.*
```

### OneLogin:

```
SAML Attribute: groups
Value: Member Of (filtered)
```

## Mapping Configuration in Dynatrace

In Account Management → Identity providers → Your IdP:

1. **Group claim name:** The attribute name from your IdP (e.g., `groups` )
2. **Group mappings:** Map each IdP group to a Dynatrace group

### Example Mappings:

| IdP Group           | Dynatrace Group     |
|---------------------|---------------------|
| IT-Dynatrace-Admins | dt-platform-admins  |
| App-Checkout-Team   | dt-checkout-editors |
| All-Employees       | dt-all-viewers      |

**Best Practice:** Use explicit mappings rather than auto-create. This prevents unauthorized groups from being provisioned.

## 5. Group Lifecycle Management

Groups have a lifecycle that requires ongoing management.

### Group Lifecycle Stages

| Stage           | Actions                          | Owner                   |
|-----------------|----------------------------------|-------------------------|
| <b>Request</b>  | Business justification, approval | Requestor               |
| <b>Create</b>   | Create group, configure policies | IAM Admin               |
| <b>Populate</b> | Add initial members              | IAM Admin / Group Owner |
| <b>Operate</b>  | Ongoing membership changes       | Group Owner             |
| <b>Review</b>   | Periodic membership audit        | Group Owner + Auditor   |
| <b>Retire</b>   | Remove members, delete group     | IAM Admin               |

### Group Owner Responsibilities

Every group should have a designated owner who:

- Approves membership requests
- Removes departed employees
- Participates in access reviews
- Maintains group description/documentation



## Membership Change Process

### Adding Members:

1. User or manager requests access
2. Group owner approves request
3. IAM admin (or automated process) adds user
4. User confirms access works

### Removing Members:

1. Triggered by: role change, termination, access review
2. Group owner authorizes removal
3. IAM admin removes user
4. Audit log captures change

## Periodic Access Reviews

| Group Type    | Review Frequency | Reviewer                 |
|---------------|------------------|--------------------------|
| Admin groups  | Monthly          | Security + Account Admin |
| Editor groups | Quarterly        | Group Owner + Manager    |
| Viewer groups | Semi-annually    | Group Owner              |

See **IAM-07: Audit Logging and Compliance** for access review automation.

## 6. Separation of Duties

---

Design your group structure to enforce separation of duties and reduce risk.

### Key Separations

| Separation           | Groups to Separate                                             | Reason                       |
|----------------------|----------------------------------------------------------------|------------------------------|
| Admin vs User        | <code>dt-platform-admins</code> vs <code>dt-all-viewers</code> | Prevent privilege escalation |
| Dev vs Prod          | <code>dt-app-dev-*</code> vs <code>dt-app-prod-*</code>        | Protect production           |
| Configure vs Operate | <code>dt-config-editors</code> vs <code>dt-operations</code>   | Change control               |

| Separation     | Groups to Separate          | Reason                |
|----------------|-----------------------------|-----------------------|
| Audit vs Admin | dt-auditors vs dt-* -admins | Independent oversight |

## Incompatible Group Pairs

Users should NOT be in both groups simultaneously:

| Group A           | Group B             | Risk                   |
|-------------------|---------------------|------------------------|
| Production Admins | Development Editors | Dev-to-prod escalation |
| Security Auditors | Platform Admins     | Self-audit             |
| Token Managers    | Application Users   | Token abuse            |

## Enforcement Options

1. **Manual Review:** Check during access reviews
2. **IdP Rules:** Configure mutually exclusive groups in IdP
3. **Automation:** Script to detect violations

## Break-Glass Groups

Emergency access should be handled through special groups:

```
dt-breakglass-admins
├─ Empty by default
├─ Temporary membership only (< 24h)
├─ Requires documented incident
└─ Automatic expiration
```

## 7. Cross-Environment Patterns

When you have multiple Dynatrace environments, design groups that work across them.

## Environment Types

| Environment    | Purpose                   | Typical Access          |
|----------------|---------------------------|-------------------------|
| Production     | Live customer data        | Restricted, audit-heavy |
| Non-Production | Dev, test, staging        | Broader access          |
| Sandbox        | Learning, experimentation | Open access             |

## Cross-Environment Group Patterns

### Pattern A: Environment-Specific Groups

```
dt-prod-checkout-admins    (prod env only)
dt-nonprod-checkout-admins (nonprod env only)
```

### Pattern B: Role Groups + Environment Assignment

```
dt-checkout-admins
├─ Assigned to Prod env with boundary X
└─ Assigned to NonProd env with boundary Y
```

### Pattern C: Tiered Groups

```
dt-tier1-admins  (all environments, full access)
dt-tier2-admins  (nonprod only, full access)
dt-tier3-viewers (all environments, view only)
```

## Recommendations

| Scenario                       | Recommended Pattern           |
|--------------------------------|-------------------------------|
| Strong prod/nonprod separation | Pattern A (env-specific)      |
| Simplified management          | Pattern B (role + assignment) |
| Clear access tiers             | Pattern C (tiered)            |

## 8. Group Assessment Queries

---

Use these queries to understand your current group usage.

```
// Review recent group membership changes
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 50
```

```
// Audit group creation events
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-90d
| filter event.kind == "AUDIT_EVENT"
| filter event.type == "CREATE" and contains(resource, "iam")
| fields timestamp, user.id, resource, event.outcome
| sort timestamp desc
| limit 50
```

```
// Find group-related changes by admin user
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| summarize changes = count(), by:{user.id, event.type}
| sort changes desc
| limit 25
```

## Next Steps

---

With your group architecture defined, proceed to configure access controls:

### Recommended Path

1. **IAM-04: Policy Authoring and Management** - Create policies for your groups
2. **IAM-05: Boundary Design Patterns** - Define what each group can see
3. **IAM-06: User Lifecycle and Provisioning** - Automate user management

**Workshop:** See **IAM-11: Policy Persona Workshop** for a guided exercise on

mapping personas to groups.

## Group Architecture Checklist

Before moving on, ensure you have:

- ☐ Chosen a group hierarchy pattern
- ☐ Defined naming conventions
- ☐ Configured SAML group mappings (if using SSO)
- ☐ Documented group ownership
- ☐ Planned separation of duties
- ☐ Designed cross-environment patterns (if applicable)

---

## Summary

In this notebook, you learned:

- Group fundamentals and properties
- Four hierarchy patterns: flat, role-based, team-based, and matrix
- Naming conventions for scalable group management
- SAML group mapping strategies
- Group lifecycle management and ownership
- Separation of duties through group design
- Cross-environment group patterns

---

## References

- [User Groups](#)
  - [SAML 2.0 Integration](#)
  - [Manage User Permissions](#)
-

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*